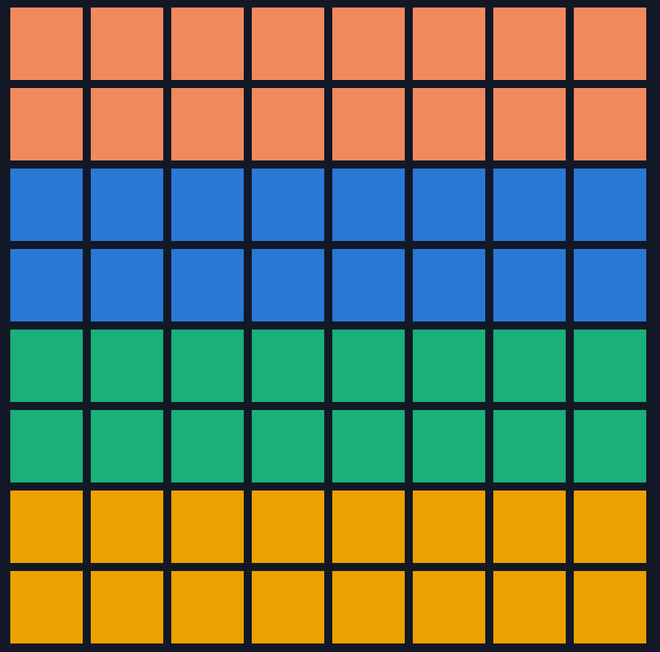
PARALLEL COMPUTING · COURSE PROJECT

Parallel Image Edge Detection

One filter, three implementations: serial C, OpenMP on CPU threads and CUDA on the GPU



Gautam Sharma · November 2026

Edge detection has to be fast

33.2M

pixels in one 8K frame (7680 × 4320)

126 ms

per 8K frame on one CPU core: about 8 frames per second, but video needs 30 to 60

Where it is used

- Self-driving cars: lanes and obstacles
- Medical scans: organ and tumour outlines
- Document scanning and OCR
- Real-time video filters

Blur first, then find the edges

1 · Input
8-bit grayscale image, HD to 8K

→

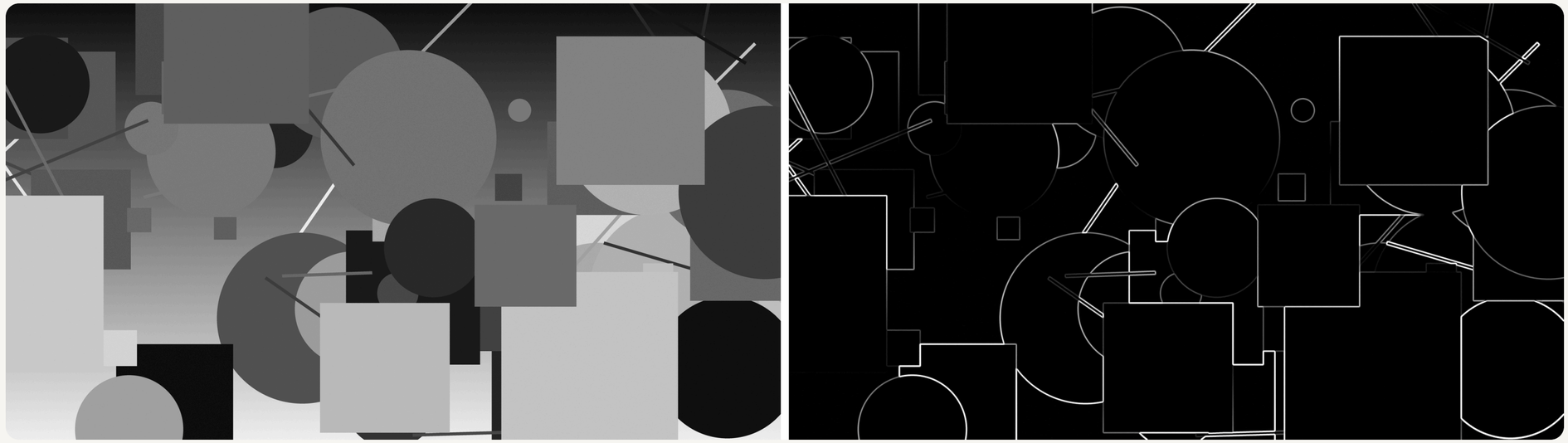
2 · Gaussian blur
5×5 weighted average removes noise

→

3 · Sobel filter
3×3 gradients gx, gy → $\sqrt{(gx^2 + gy^2)}$

→

4 · Edge map
Bright pixel = strong edge



Left: input · Right: our edge-detection output (Full HD test image)

Three versions of the same filter

Each output pixel depends only on its neighbours in the input, so all 33 million can be computed at the same time: **data parallelism**.

Version	Runs on	How the work is split	Syllabus
Serial	1 CPU core	Double loop over every pixel	Baseline
OpenMP	10 CPU cores (M5)	Image rows shared among threads	Module 4
CUDA naive	T4 GPU, 2,560 cores	One thread per pixel, 16×16 blocks	Module 5
CUDA shared memory	T4 GPU, 2,560 cores	Each block caches its tile on-chip	Modules 2, 5

OpenMP: one extra line

```
#pragma omp parallel for schedule(runtime)
for (int y = 0; y < h; y++)
    for (int x = 0; x < w; x++)
        out[y*w + x] = blur_pixel(in, w, h, x, y);
```

The serial version is the same loop without the orange line. Each thread gets a share of the rows (y values).

SHARED

Input and output images, width, height

PRIVATE

Loop counters x and y: one copy per thread

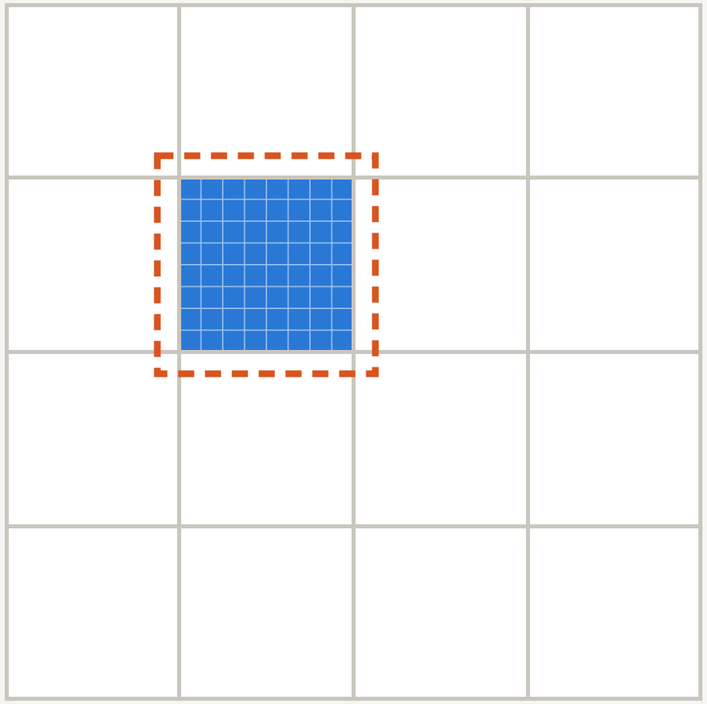
NO LOCKS NEEDED

Each thread writes different pixels: no race, no reduction

SCHEDULE(RUNTIME)

Static or dynamic, picked with OMP_SCHEDULE

CUDA: one thread per pixel



One block: 16×16 threads, one per pixel

Tile + 2-pixel halo (20×20) copied to shared memory

1. Thread to pixel: x = blockIdx.x × blockDim.x + threadIdx.x (same for y)
2. **Naive kernel:** every thread reads its 25 neighbours from slow global memory, so each pixel is fetched up to 25 times
3. **Shared-memory kernel:** the block copies its tile and halo once into fast on-chip memory, then waits at __syncthreads()
4. Blur and Sobel are separate kernels: the launch boundary is a barrier across the whole GPU

How we measured

CPU

Apple M5, 10 cores (4 fast + 6 efficiency), clang -O2, 1–10 threads

GPU

NVIDIA Tesla T4 (Google Colab), 2,560 CUDA cores, nvcc -O2

IMAGES

HD, Full HD, 4K and 8K: from 0.9 to 33.2 million pixels

TIMING

Best of 5 runs; CPU: clock_gettime, GPU: cudaEvent (copies timed apart)

Verified: every parallel output matches the serial output, pixel for pixel

LIVE DEMO

Let’s run it

PART 1 · CPU · LAPTOP TERMINAL

`bash demo.sh`

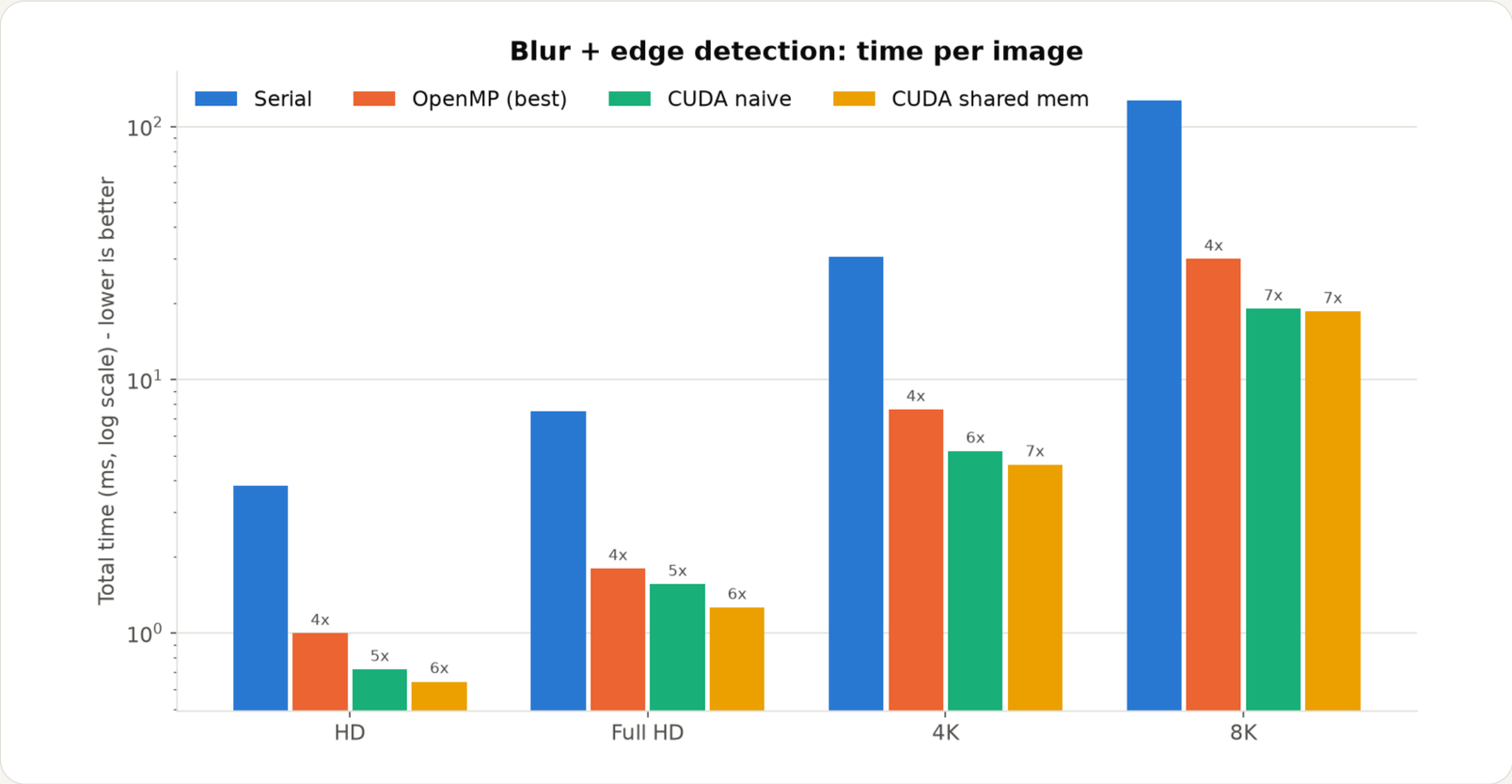
- Serial on the 8K image
- OpenMP on 1 → 10 threads
- Static vs dynamic scheduling
- Pixel-by-pixel correctness check

PART 2 · GPU · GOOGLE COLAB (T4)

`Section 10`

- Naive vs shared-memory kernels
- Copy time vs compute time
- GPU output checked against serial
- The edge map, displayed

The GPU wins, OpenMP close behind



Log scale · CPU: Apple M5 · GPU: NVIDIA T4, including data copies

8K IMAGE

126 ms

Serial, 1 core

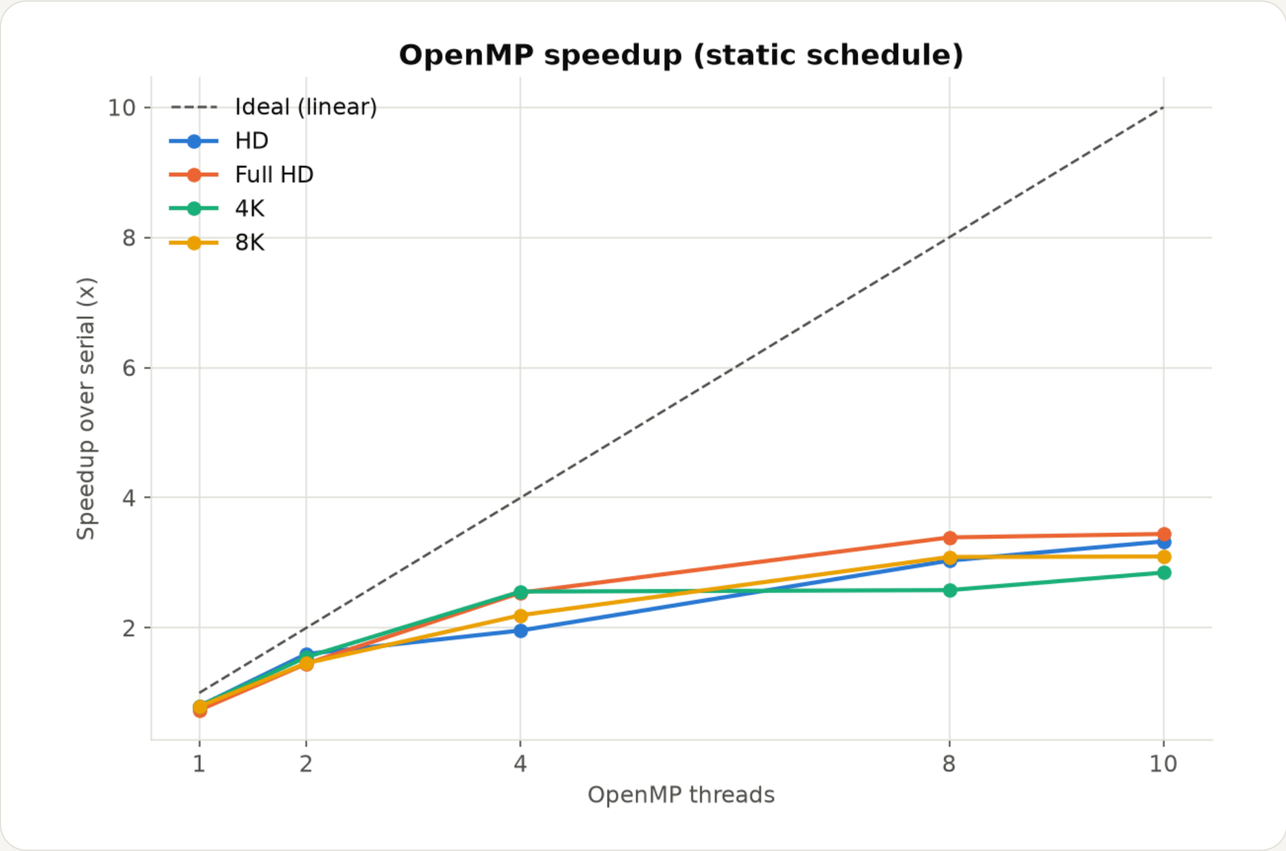
30 ms

OpenMP, 10 threads: 4.2× faster

18.6 ms

CUDA with copies: 6.8× faster

OpenMP speedup flattens after 4 threads



schedule(static) · dashed line = ideal linear speedup

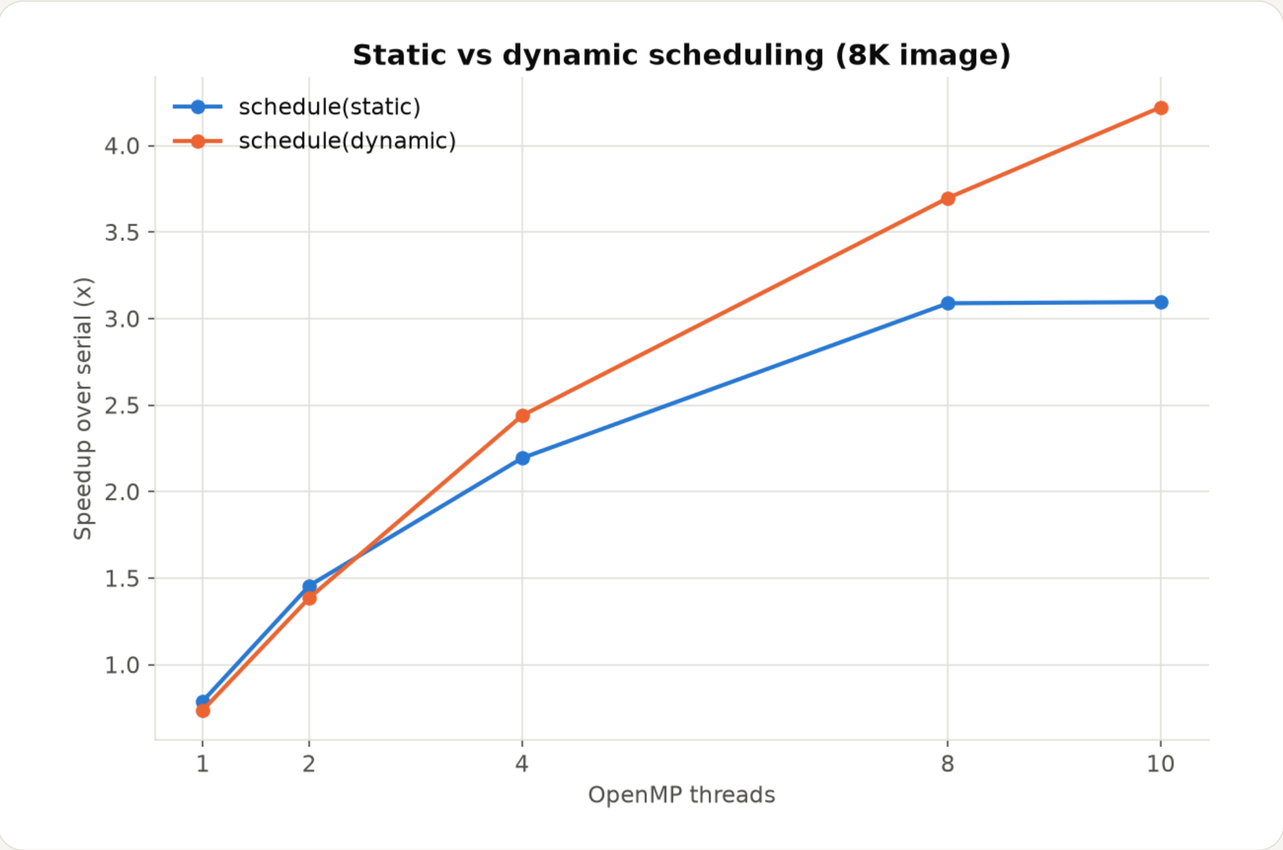
Why not 10×?

Mixed cores: only 4 of the 10 are fast performance cores

Memory-bound: all threads share the same memory bandwidth

Overhead: OpenMP on 1 thread is 27% slower than serial

Dynamic scheduling beats static



8K image, Apple M5

40.8 → 29.9 ms

10 threads: speedup 3.1× → 4.2×

SCHEDULE(STATIC)

Equal rows per thread: fast cores finish early and wait for slow ones. **Load imbalance.**

SCHEDULE(DYNAMIC)

Rows handed out on demand, so fast cores simply take more.

Amdahl’s law sets the ceiling

$$S(p) = 1 / (f + (1 - f) / p)$$

0.15

measured serial fraction f (Karp–Flatt, 8K, 10 threads)

4.2×

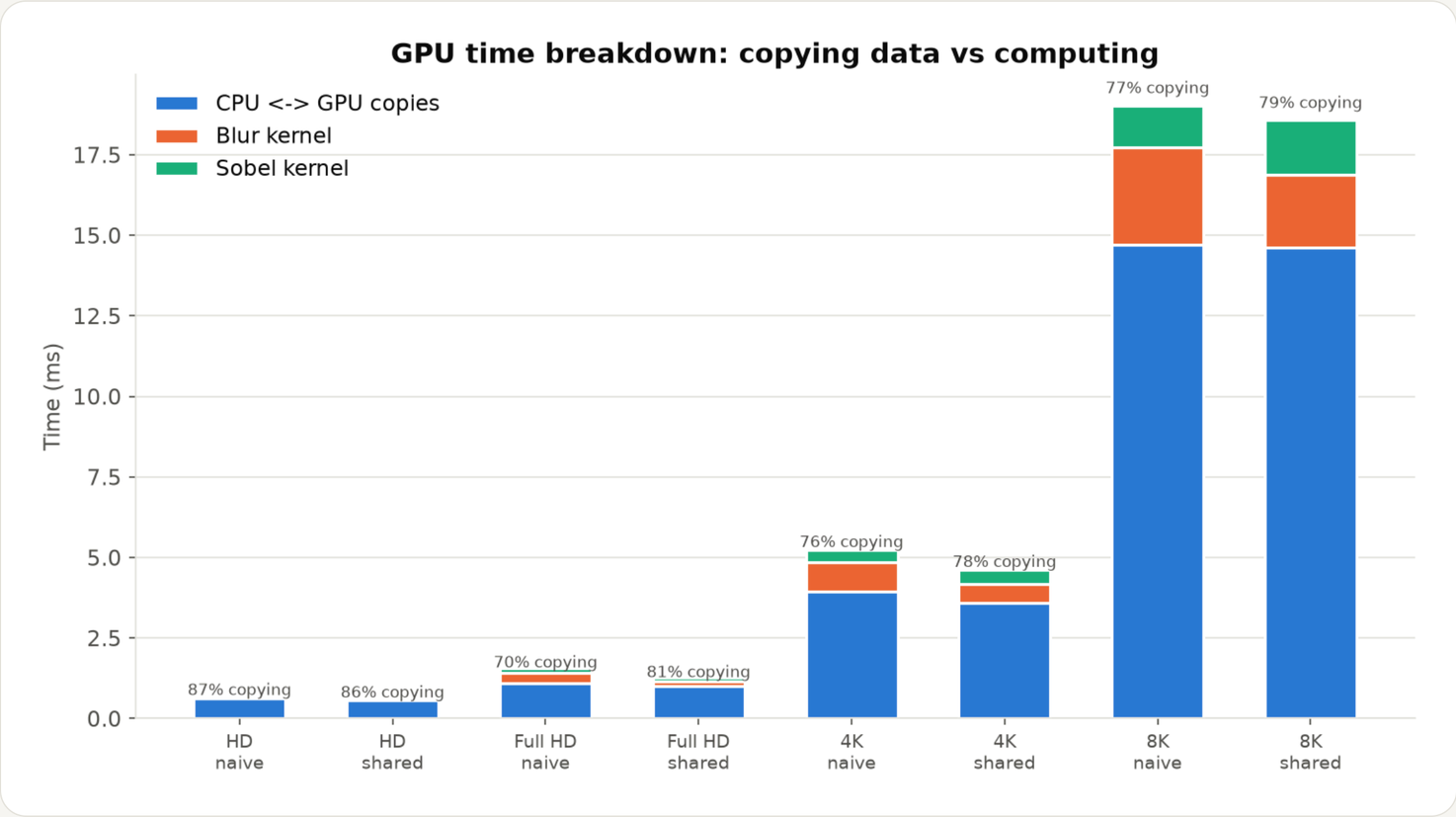
speedup we got on 10 threads

6.6×

best possible even with infinite cores (1 / f)

Our f is measured, so besides truly serial work it also includes memory-bandwidth limits and the slower efficiency cores.

GPU time goes to copying data



NVIDIA T4 on Google Colab

79%

of GPU time on 8K is spent copying the image

32× faster

than serial: kernels alone (4.0 ms)

6.8× faster

than serial: with copies (18.6 ms)

Shared memory helped blur, not Sobel

Kernel (8K image)	Naive	Shared memory	Change
Gaussian blur, 5×5	3.03 ms	2.26 ms	26% faster
Sobel, 3×3	1.29 ms	1.71 ms	32% slower

Blur: lots of reuse

Each pixel is read by 25 threads. Loading it once into shared memory saves many slow reads.

Sobel: little reuse

Only 9 reads per pixel, and the GPU cache already helps. The extra copy and __syncthreads() cost more.

What we learned

01

Data-parallel work scales: one pragma gave **4.2×** on 10 CPU cores

02

Hardware matters: on mixed cores, dynamic scheduling cut time by **27%**

03

The GPU computes **32×** faster than one core, but copying takes about 80% of its time

04

Optimise for reuse: shared memory helped the 5×5 blur and hurt the 3×3 Sobel

FUTURE WORK

- Pinned memory and CUDA streams to overlap copies with computing
- Separable blur: 10 multiplications per pixel instead of 25
- Keep video frames on the GPU through a whole pipeline
- MPI to spread video frames across several machines

Thank you

Questions?